LAPORAN PRAKTIKUM

POSTTEST (7)

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Athasyahri Syawal Farezy (2509106045)

Kelas (A2‘25)

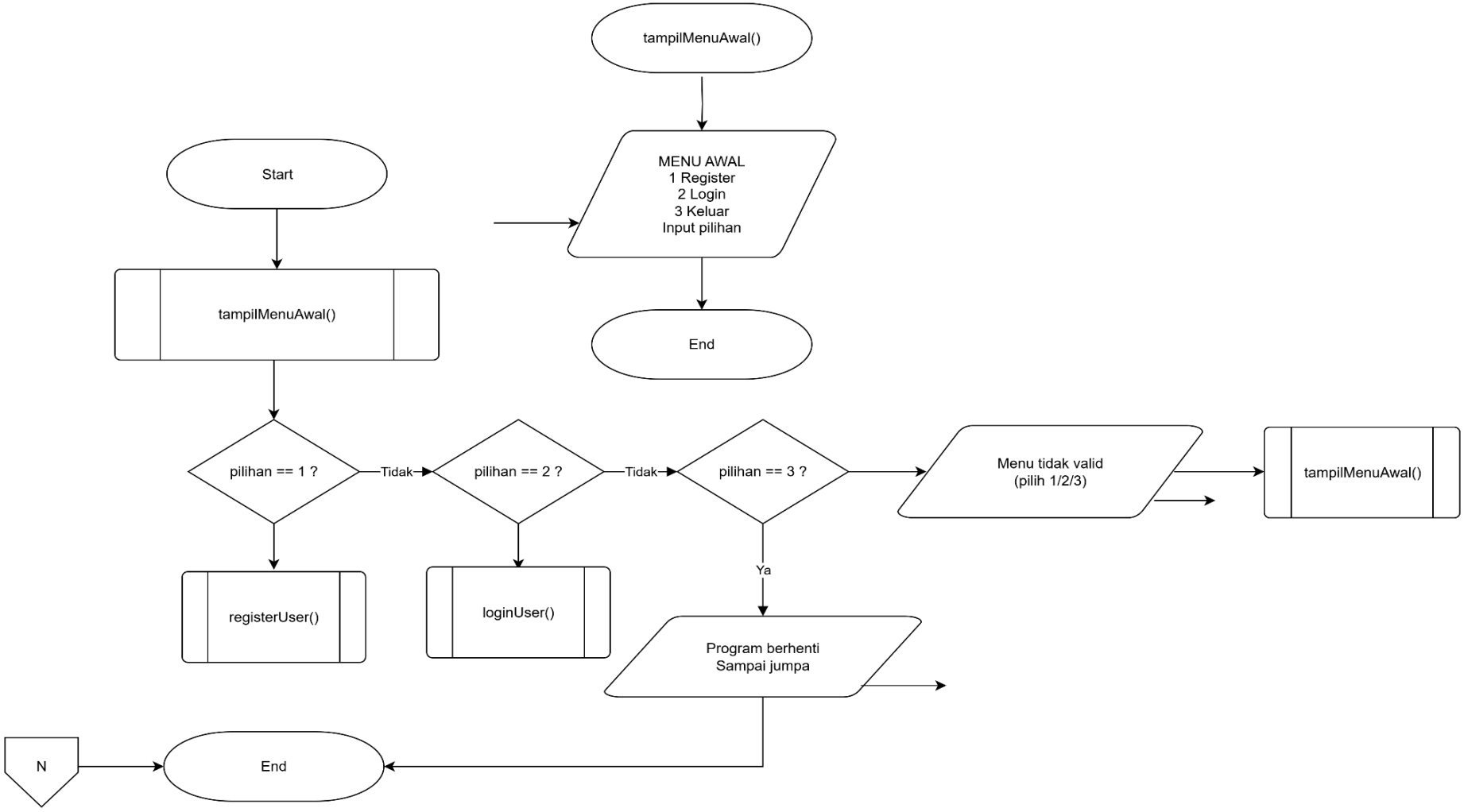
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

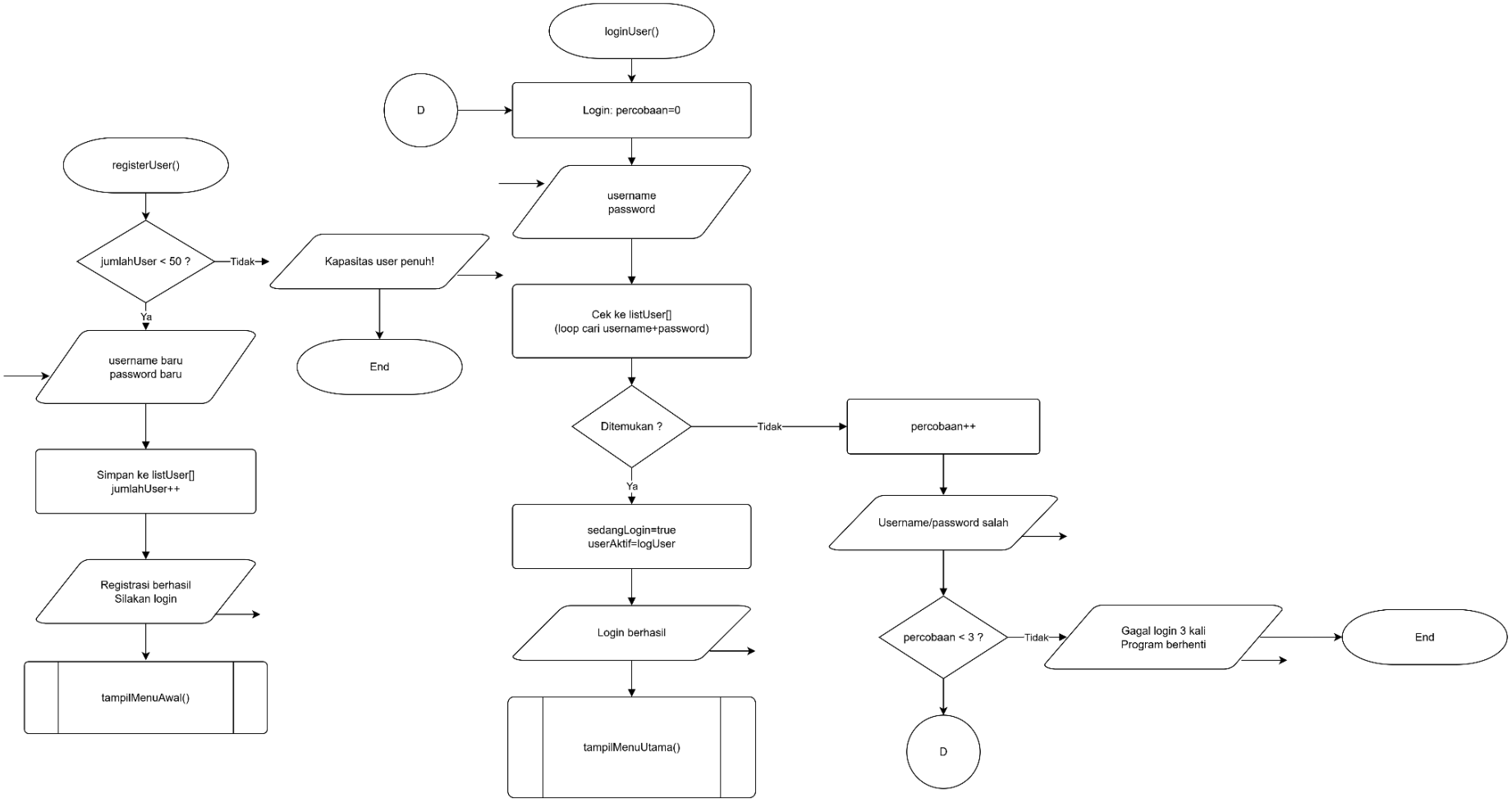
SAMARINDA

2025

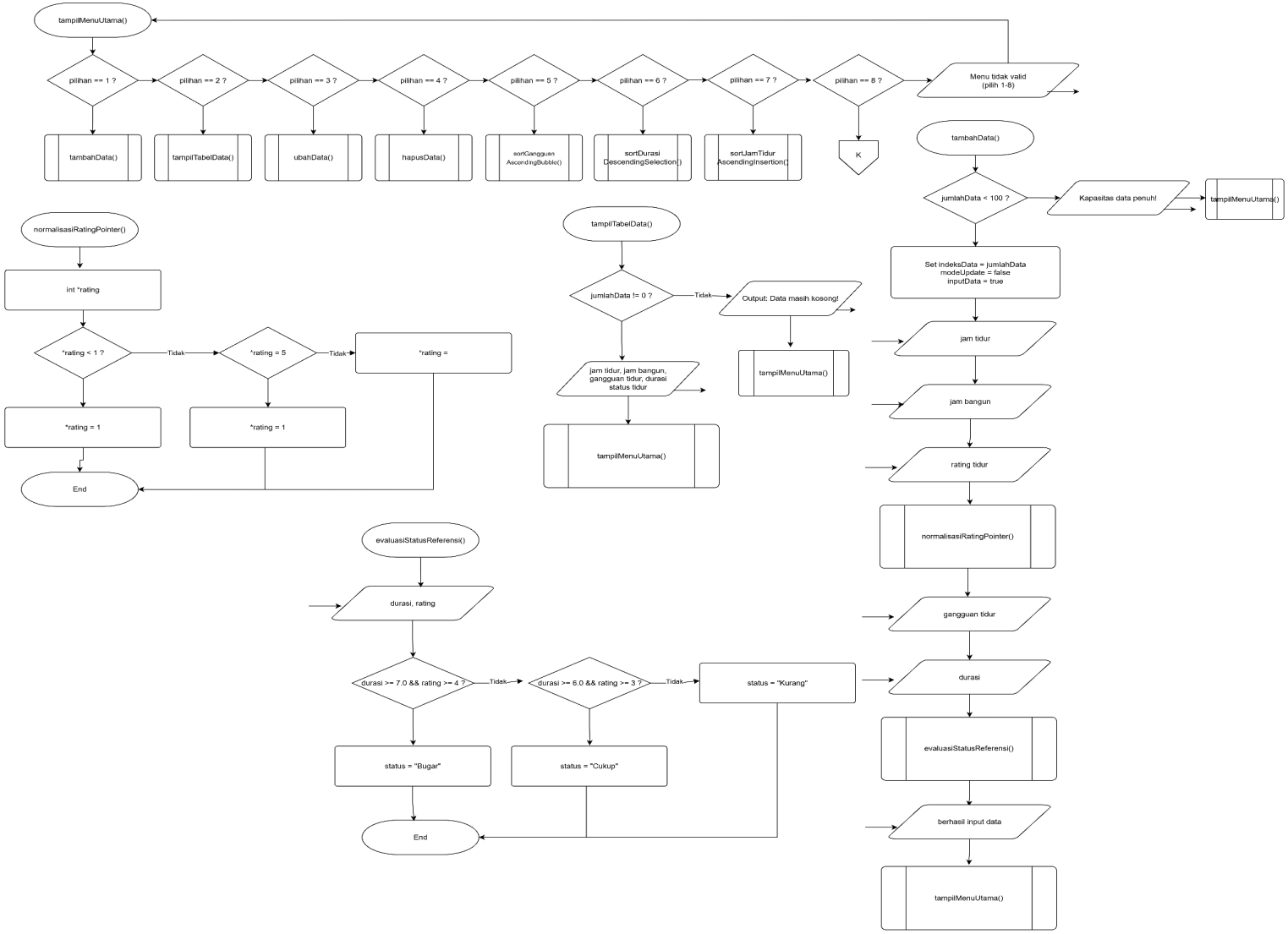
1. Flowchart



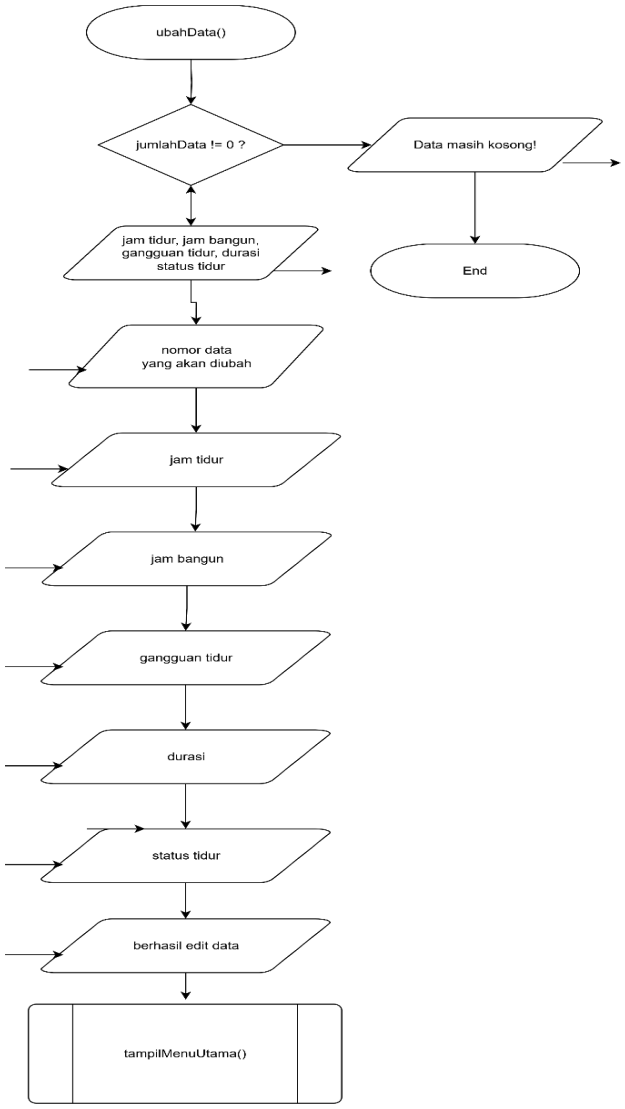
Gambar 1.1 Flowchart (1)

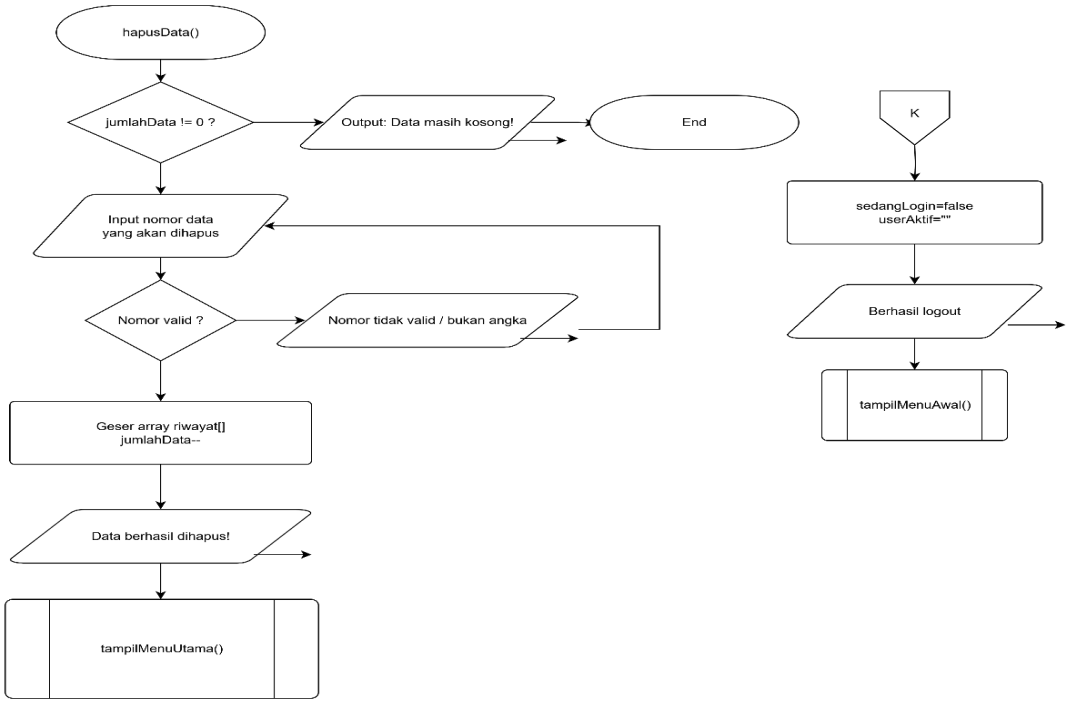


Gambar 1.2 Flowchart (2)

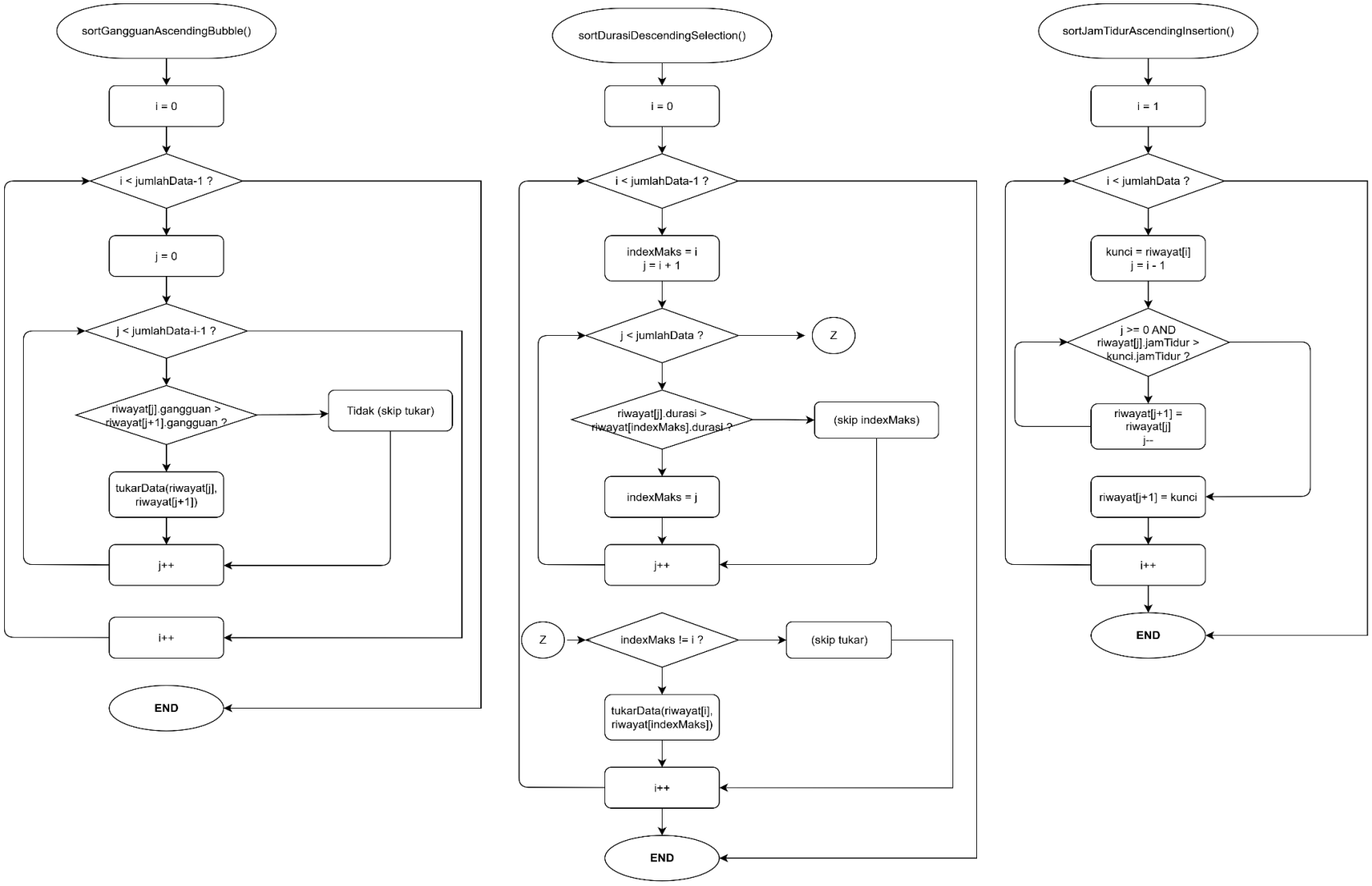


Gambar 1.3 Flowchart (3)

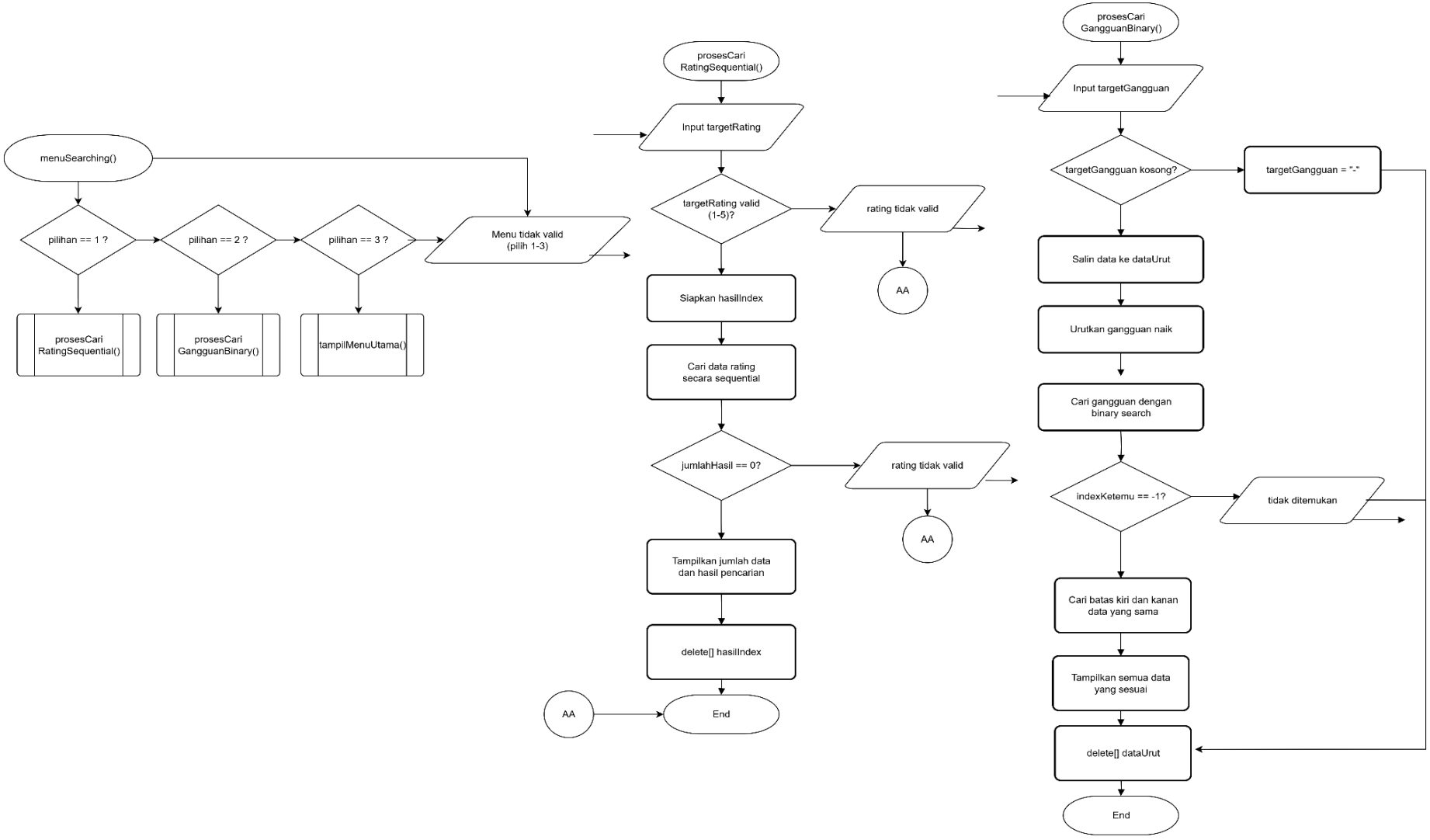




Gambar 1.4 Flowchart (4)



Gambar 1.5 Flowchart (5)



Gambar 1.6 Flowchart (6)

2. Deskripsi Singkat Program

Program ini berfungsi sebagai buku catatan digital untuk melacak kebiasaan tidur yang dilengkapi dengan sistem akun untuk setiap penggunanya. Di dalamnya, kamu bisa bebas mengelola data dengan fitur utama yang lengkap: mulai dari menambahkan catatan tidur baru (jam tidur, bangun, dan keluhan), melihat riwayat tidur yang tersusun rapi dalam bentuk tabel, mengubah kembali data jika ada kesalahan input, hingga menghapus catatan yang sudah tidak diperlukan. Seluruh aktivitas ini dilindungi oleh sistem login, sehingga setiap pengguna memiliki privasi atas riwayat tidurnya masing-masing.

3. Source Code

A. Deklarasi Struct

Section ini merupakan bagian deklarasi beberapa struct yang dibutuhkan.

Source Code:

```
struct WaktuDetail // nested struct nya
{
    string jamTidur;
    string jamBangun;
};

struct SleepTracker
{
    WaktuDetail waktu; // dimasukkan kesini
    int rating;
    string gangguan;
    float durasi;
    string status;
};

struct User
{
    string username;
    string password;
};
```

B. Fitur Register

Section ini merupakan proses registerasi. Input username dan password nya.

Source Code:

```
void registerUser(Uservoid registerUser(User listUser[], int &jumlahUser,
int maxUser)
{
    if (jumlahUser >= maxUser)
    {
        cout << "Kapasitas user penuh!" << endl;
        return;
    }

    cout << "\n--- REGISTER ---" << endl;
    cout << "Masukkan Username Baru: ";
    cin >> listUser[jumlahUser].username;
    cout << "Masukkan Password Baru: ";
    cin >> listUser[jumlahUser].password;
    jumlahUser++;
    cout << "Registrasi Berhasil! Silahkan Login." << endl;
           }listUser[], int &jumlahUser, int maxUser)
{
    if (jumlahUser >= maxUser)
    {
        cout << "Kapasitas user penuh!" << endl;
        return;
    }

    cout << "\n--- REGISTER ---" << endl;
    cout << "Masukkan Username Baru: ";
    cin >> listUser[jumlahUser].username;
    cout << "Masukkan Password Baru: ";
    cin >> listUser[jumlahUser].password;
    jumlahUser++;
    cout << "Registrasi Berhasil! Silahkan Login." << endl;
}
```

C. Fitur Login

Section ini merupakan proses autentikasi. Menggunakan perulangan while untuk membatasi percobaan akses maksimal 3 kali.

Source Code:

```
bool loginUser(const User listUser[], int jumlahUser, string &userAktif)
{
    string username;
    string password;
    int percobaan = 0;

    while (percobaan < 3)
    {
        cout << "\n--- LOGIN (Percobaan " << percobaan + 1 << "/3) ---" <<
                 endl;
        cout << "Username: ";
        cin >> username;
        cout << "Password: ";
        cin >> password;

        for (int i = 0; i < jumlahUser; i++)
        {
            if (listUser[i].username == username && listUser[i].password ==
                password)
            {
                userAktif = username;
                cout << "Login Berhasil! Halo, " << userAktif << endl;
                return true;
            }
        }

        percobaan++;
        cout << "Username atau password salah!" << endl;
    }

    return false;
}
```

D. Fitur Menu Utama (Pilih Menu)

Bagian ini berfungsi sebagai pusat kendali interaktif yang memungkinkan pengguna memilih berbagai fitur utama aplikasi, seperti menambah, melihat, mengubah, atau menghapus data. Bakalan diulang terus sampai logout

Source Code:

```
void tampilMenuUtama(const string &userAktif)
{
    cout << "\n=== APLIKASI SLEEP TRACKER (User: " << userAktif << ") ==="
<< endl;
    cout << "1. Tambah Catatan Tidur (Create)" << endl;
    cout << "2. Lihat Riwayat Tidur (Read)" << endl;
    cout << "3. Ubah Catatan (Update)" << endl;
    cout << "4. Hapus Catatan (Delete)" << endl;
    cout << "5. Sorting Huruf Ascending (Gangguan A-Z) [Bubble Sort]" <<
            endl;
    cout << "6. Sorting Angka Descending (Durasi besar-kecil) [Selection
            Sort]" << endl;
    cout << "7. Sorting Bebas (Jam Tidur awal-akhir) [Insertion Sort]" <<
            endl;
    cout << "8. Searching Data" << endl;
    cout << "9. Logout" << endl;
}
```

E. Fitur Create

Fitur Create berfungsi untuk memasukkan data tidur ke dalam sistem dengan validasi agar input tetap benar. Pada fungsi inputDataTidur(SleepTracker *data), digunakan pointer ke struct sehingga data asli dapat langsung diubah dengan operator ->. Program memeriksa format jam, rating, gangguan, dan durasi tidur, lalu memanggil evaluasiStatusReferensi(*data) untuk menentukan status tidur secara otomatis.

Source Code:

```
void inputDataTidur(SleepTracker *data)
{
    string pesanError;

    while (true)
    {
```

```
        try
        {
            cout << "Jam Tidur (HH:MM): ";
            cin >> data->waktu.jamTidur;

            if (!validJam(data->waktu.jamTidur, pesanError))
            {
                throw invalid_argument(pesanError);
            }

            break;
        }
        catch (const exception &e)
        {
            cout << e.what() << endl;
        }
    }

    while (true)
    {
        try
        {
            cout << "Jam Bangun (HH:MM): ";
            cin >> data->waktu.jamBangun;

            if (!validJam(data->waktu.jamBangun, pesanError))
            {
                throw invalid_argument(pesanError);
            }

            break;
        }
        catch (const exception &e)
        {
            cout << e.what() << endl;
        }
    }

    while (true)
    {
        try
        {
            cout << "Rating Kualitas (1-5): ";
            cin >> data->rating;

            if (cin.fail())
            {
                cin.clear();
```

```
            cin.ignore(10000, '\n');
            throw invalid_argument("Rating harus angka 1 sampai 5.");
        }

        if (data->rating < 1 || data->rating > 5)
        {
            throw out_of_range("Rating hanya boleh 1 sampai 5.");
        }

        break;
    }
    catch (const exception &e)
    {
        cout << e.what() << endl;
    }
}

normalisasiRatingPointer(&data->rating);

cout << "Gangguan: ";
cin.ignore(10000, '\n');
getline(cin, data->gangguan);
if (data->gangguan.empty())
{
    data->gangguan = "-";
}

while (true)
{
    try
    {
        cout << "Total Durasi (Jam): ";
        cin >> data->durasi;

        if (cin.fail())
        {
            cin.clear();
            cin.ignore(10000, '\n');
            throw invalid_argument("Durasi harus angka (contoh: 7 atau
                  7.5).");
        }

        if (data->durasi <= 0)
        {
            throw out_of_range("Durasi harus lebih dari 0.");
        }
```

```
            break;
        }
        catch (const exception &e)
        {
            cout << e.what() << endl;
        }
    }

    evaluasiStatusReferensi(*data);
    cout << "Status Tidur Otomatis: " << data->status << endl;
}
```

F. Fitur Read

Bagian ini berfungsi untuk menampilkan seluruh riwayat tidur yang telah tersimpan ke dalam format tabel yang rapi dan terorganisir menggunakan pustaka iomanip agar data mudah dibaca oleh pengguna.

Source Code:

```
void tampilTabelData(const SleepTracker riwayat[], int jumlahData)
{
    if (jumlahData == 0)
    {
        cout << "\nData masih kosong!" << endl;
        return;
    }

    const int lebarTabel = 90;
    cout << "\n"
         << setfill('=') << setw(lebarTabel) << "" << endl;
    cout << setfill(' ') << left << setw(5) << "NO"
         << setw(18) << "WAKTU"
         << setw(9) << "RATING"
         << setw(15) << "DURASI(JAM)"
         << setw(12) << "STATUS"
         << "GANGGUAN" << endl;
    cout << setfill('-') << setw(lebarTabel) << "" << endl;
    cout << fixed << setprecision(1);

    tampilBarisDataRekursif(riwayat, 0, jumlahData);

    cout << setfill('=') << setw(lebarTabel) << "" << endl;
    cout << setfill(' ');
}
```

G. Fitur Update

Fitur ini memungkinkan pengguna untuk memperbarui data yang salah dengan cara memilih nomor urut tertentu dari tabel riwayat, kemudian menggantinya dengan informasi baru melalui proses validasi yang sama seperti saat menambah data.

Source Code:

```
void ubahData(SleepTracker riwayat[], int jumlahData)
{
    if (jumlahData == 0)
    {
        cout << "\nData masih kosong!" << endl;
        return;
    }

    cout << "\n--- Ubah Data ---" << endl;
    tampilTabelData(riwayat, jumlahData);
    int index = pilihIndexData(jumlahData, "Masukkan nomor data yang ingin
                diubah: ");
    cout << "Masukkan data pengganti:" << endl;
    inputDataTidur(riwayat[index]);
    cout << "Data berhasil diubah!" << endl;
}
```

H. Fitur Delete

Fitur ini memungkinkan pengguna untuk menghapus catatan tertentu berdasarkan nomor urutnya, di mana sistem akan menggeser data yang ada untuk mengisi celah yang kosong dan memperbarui total jumlah penyimpanan secara otomatis.

Source Code:

```
void hapusData(SleepTracker riwayat[], int &jumlahData)
{
    if (jumlahData == 0)
    {
```

```
        cout << "\nData masih kosong!" << endl;
        return;
    }


    int index = pilihIndexData(jumlahData, "Masukkan nomor data yang ingin
                dihapus: ");

    for (int i = index; i < jumlahData - 1; i++)
    {
        riwayat[i] = riwayat[i + 1];
    }

    jumlahData--;
    cout << "Data berhasil dihapus!" << endl;
}
```

I. Fitur Logout

Fitur ini berfungsi untuk mengakhiri sesi aktif pengguna dengan mereset status login dan mengosongkan nama pengguna yang terdaftar, sehingga aplikasi kembali ke menu awal untuk menjaga keamanan data.

Source Code:

```
else
{
    sedangLogin = false;
    userAktif = "";
    cout << "Berhasil logout." << endl;
}
```

J. Fitur Pointer Dereference (*)

Fitur Pointer Dereference (*) pada kode tersebut digunakan untuk mengakses dan mengubah langsung nilai variabel rating melalui alamat memorinya. Fungsi normalisasiRatingPointer(int *rating) menerima parameter berupa pointer ke variabel rating, lalu melakukan pengecekan nilai. Jika nilai rating kurang dari 1, maka akan diubah menjadi 1. Jika nilai rating lebih dari 5, maka akan diubah menjadi 5. Dengan begitu, fungsi ini memastikan bahwa nilai rating tetap berada dalam batas yang valid, yaitu 1 sampai 5. Pointer dereference ditunjukkan pada penulisan *rating, yang berarti mengambil atau mengubah nilai asli dari variabel yang ditunjuk pointer tersebut.

Source Code:

```
void normalisasiRatingPointer(int *rating)
{
    if (*rating < 1)
    {
        *rating = 1;
    }
    else if (*rating > 5)
    {
        *rating = 5;
    }
}
```

K. Fitur Pointer Address-of / reference (&)

Fitur Pointer Address-of / Reference (&) pada fungsi evaluasiStatusReferensi(SleepTracker &data) digunakan untuk mengirim struct SleepTracker secara referensi, sehingga perubahan pada data.status langsung terjadi pada data asli tanpa membuat salinan baru. Fungsi ini mengevaluasi kondisi berdasarkan durasi tidur dan rating, lalu otomatis memberi status “Bugar”, “Cukup”, atau “Kurang” sesuai nilai yang dimasukkan.

Source Code:

```
void evaluasiStatusReferensi(SleepTracker &data)
{
    if (data.durasi >= 7.0f && data.rating >= 4)
    {
        data.status = "Bugar";
    }
    else if (data.durasi >= 6.0f && data.rating >= 3)
    {
        data.status = "Cukup";
    }
    else
    {
        data.status = "Kurang";
    }
}
```

L. Bubble Sort Sorting Huruf Secara Ascending

Fungsi sortGangguanAscendingBubble mengurutkan array riwayat berdasarkan field gangguan dari nilai terkecil ke terbesar menggunakan algoritma Bubble Sort dengan dua loop bersarang, di mana setiap iterasi membandingkan elemen berdekatan dan menukarnya via fungsi tukarData jika elemen kiri lebih besar dari elemen kanan.

Source Code:

```
void sortGangguanAscendingBubble(SleepTracker riwayat[], int jumlahData)
{
    for (int i = 0; i < jumlahData - 1; i++)
    {
        for (int j = 0; j < jumlahData - i - 1; j++)
        {
            if (riwayat[j].gangguan > riwayat[j + 1].gangguan)
            {
                tukarData(riwayat[j], riwayat[j + 1]);
            }
        }
    }
}
```

M. Selection Sort Sorting Angka Secara Descending

Fungsi sortDurasiDescendingSelection mengurutkan array riwayat berdasarkan field durasi dari nilai terbesar ke terkecil menggunakan algoritma Selection Sort, di mana setiap iterasi outer loop mencari indeks elemen maksimum dari sisa array melalui inner loop, lalu menukarnya dengan posisi i menggunakan tukarData jika indeks maksimum berbeda dari posisi saat ini.

Source Code:

```
void sortDurasiDescendingSelection(SleepTracker riwayat[], int jumlahData)
{
    for (int i = 0; i < jumlahData - 1; i++)
    {
        int indexMaks = i;

        for (int j = i + 1; j < jumlahData; j++)
        {
            if (riwayat[j].durasi > riwayat[indexMaks].durasi)
            {
                indexMaks = j;
            }
        }

        if (indexMaks != i)
        {
            tukarData(riwayat[i], riwayat[indexMaks]);
        }
    }
}
```

M. Insertion Sort Sorting Angka Ascending

Fungsi sortJamTidurAscendingInsertion mengurutkan array riwayat berdasarkan field waktu.jamTidur dari nilai terkecil ke terbesar menggunakan algoritma Insertion Sort, di mana setiap iterasi mengambil elemen ke-i sebagai kunci lalu menggeser elemen-elemen sebelumnya yang lebih besar ke kanan hingga ditemukan posisi yang tepat untuk menyisipkan kunci tersebut.

Source Code:

```
void sortJamTidurAscendingInsertion(SleepTracker riwayat[], int jumlahData)
{
    for (int i = 1; i < jumlahData; i++)
    {
        SleepTracker kunci = riwayat[i];
        int j = i - 1;

        while (j >= 0 && riwayat[j].waktu.jamTidur > kunci.waktu.jamTidur)
        {
            riwayat[j + 1] = riwayat[j];
            j--;
        }

        riwayat[j + 1] = kunci;
    }
}
```

M. Insertion Sort Sorting Angka Ascending

Fungsi sortJamTidurAscendingInsertion mengurutkan array riwayat berdasarkan field waktu.jamTidur dari nilai terkecil ke terbesar menggunakan algoritma Insertion Sort, di mana setiap iterasi mengambil elemen ke-i sebagai kunci lalu menggeser elemen-elemen sebelumnya yang lebih besar ke kanan hingga ditemukan posisi yang tepat untuk menyisipkan kunci tersebut.

Source Code:

```
void sortJamTidurAscendingInsertion(SleepTracker riwayat[], int jumlahData)
{
    for (int i = 1; i < jumlahData; i++)
    {
        SleepTracker kunci = riwayat[i];
        int j = i - 1;

        while (j >= 0 && riwayat[j].waktu.jamTidur > kunci.waktu.jamTidur)
        {
            riwayat[j + 1] = riwayat[j];
            j--;
        }

        riwayat[j + 1] = kunci;
    }
}
```

M. Binary Search Pencarian Data Gangguan Tidur

Fungsi cariGangguanBinary digunakan untuk mencari data pada array riwayatUrut berdasarkan field gangguan menggunakan algoritma Binary Search. Sebelum proses pencarian dilakukan, fungsi prosesCariGangguanBinary terlebih dahulu menyalin data asli ke array baru dataUrut, lalu mengurutkannya secara ascending berdasarkan field gangguan dengan fungsi sortGangguanAscendingBubble, karena Binary Search hanya dapat diterapkan pada data yang sudah terurut. Setelah itu, pengguna memasukkan nilai targetGangguan yang ingin dicari. Jika input kosong, maka nilai otomatis diubah menjadi "-".

Pada fungsi cariGangguanBinary, pencarian dilakukan dengan menentukan batas kiri (kiri) dan kanan (kanan), lalu menghitung posisi tengah (tengah). Nilai gangguan pada elemen tengah dibandingkan dengan targetGangguan. Jika sama, fungsi mengembalikan indeks data yang ditemukan. Jika nilai tengah lebih kecil dari target, pencarian dilanjutkan ke bagian kanan array, sedangkan jika lebih besar, pencarian dilanjutkan ke bagian kiri. Jika data tidak ditemukan, fungsi mengembalikan -1.

Selanjutnya, pada fungsi prosesCariGangguanBinary, jika data ditemukan maka program tidak hanya menampilkan satu data saja, tetapi juga menelusuri ke kiri dan ke kanan dari indeks yang ditemukan untuk mencari semua data lain yang memiliki nilai gangguan yang sama. Dengan demikian, seluruh data dengan gangguan tidur yang sama dapat ditampilkan. Jika tidak ditemukan, program akan menampilkan pesan bahwa data gangguan tersebut tidak ada.

Source Code:

```
int cariGangguanBinary(const SleepTracker *riwayatUrut, int jumlahData,
const string *targetGangguan)
{
    int kiri = 0;
    int kanan = jumlahData - 1;

    while (kiri <= kanan)
    {
        int tengah = (kiri + kanan) / 2;
        string nilaiTengah = (riwayatUrut + tengah)->gangguan;

        if (nilaiTengah == *targetGangguan)
        {
            return tengah;
        }
        else if (nilaiTengah < *targetGangguan)
        {
            kiri = tengah + 1;
        }
        else
        {
            kanan = tengah - 1;
        }
    }

    return -1;
}
```

```
void prosesCariGangguanBinary(const SleepTracker riwayat[], int jumlahData)
{
    string targetGangguan;
    cout << "Masukkan gangguan yang dicari (harus sama persis): ";
    cin.ignore(10000, '\n');
    getline(cin, targetGangguan);

    if (targetGangguan.empty())
    {
        targetGangguan = "-";
    }

    SleepTracker *dataUrut = new SleepTracker[jumlahData];
    for (int i = 0; i < jumlahData; i++)
    {
        dataUrut[i] = riwayat[i];
    }

    sortGangguanAscendingBubble(dataUrut, jumlahData);
    int indexKetemu = cariGangguanBinary(dataUrut, jumlahData,
                      &targetGangguan);

    if (indexKetemu == -1)
    {
        cout << "Gangguan \"" << targetGangguan << "\" tidak ditemukan." <<
                endl;
    }
    else
    {
        int kiri = indexKetemu;
        int kanan = indexKetemu;

        while (kiri > 0 && dataUrut[kiri - 1].gangguan == targetGangguan)
        {
            kiri--;
        }

        while (kanan < jumlahData - 1 && dataUrut[kanan + 1].gangguan ==
               targetGangguan)
        {
            kanan++;
        }

        cout << "Gangguan \"" << targetGangguan << "\" ditemukan pada " <<
                (kanan - kiri + 1) << " data:" << endl;
```

```
        for (int i = kiri; i <= kanan; i++)
        {
            tampilDataSingkat(dataUrut[i], i + 1);
        }
    }

    delete[] dataUrut;
}
```

M. Cari Angka (Rating) [Sequential Search]

Fungsi cariGangguanBinary digunakan untuk mencari data pada array riwayatUrut berdasarkan field gangguan menggunakan algoritma Binary Search. Proses pencarian dilakukan dengan membagi area pencarian menjadi dua bagian secara berulang, lalu membandingkan nilai tengah dengan targetGangguan. Jika nilai tengah sama dengan target, maka fungsi mengembalikan indeks data tersebut. Jika nilai tengah lebih kecil dari target, pencarian dilanjutkan ke bagian kanan array, sedangkan jika lebih besar maka pencarian dilanjutkan ke bagian kiri. Apabila data tidak ditemukan, fungsi akan mengembalikan nilai -1.

Source Code:

```
int cariRatingSequential(const SleepTracker *riwayat, int jumlahData, int
targetRating, int *hasilIndex, int maxHasil)
{
    int jumlahHasil = 0;

    for (int i = 0; i < jumlahData; i++)
    {
        if ((riwayat + i)->rating == targetRating && jumlahHasil < maxHasil)
        {
            *(hasilIndex + jumlahHasil) = i;
            jumlahHasil++;
        }
    }

    return jumlahHasil;
}
```

```
void prosesCariRatingSequential(const SleepTracker riwayat[], int
jumlahData, int maxData)
{
    int targetRating = 0;

    while (true)
    {
        try
        {
            cout << "Masukkan rating yang dicari (1-5): ";
            cin >> targetRating;

            if (cin.fail())
            {
                cin.clear();
                cin.ignore(10000, '\n');
                throw invalid_argument("Input rating harus angka.");
            }

            if (targetRating < 1 || targetRating > 5)
            {
                throw out_of_range("Rating hanya boleh 1 sampai 5.");
            }

            break;
        }
        catch (const exception &e)
        {
            cout << e.what() << endl;
        }
    }

    int *hasilIndex = new int[maxData];
    int jumlahHasil = cariRatingSequential(riwayat, jumlahData,
targetRating, hasilIndex, maxData);

    if (jumlahHasil == 0)
    {
        cout << "Data dengan rating " << targetRating << " tidak ditemukan."
<< endl;
    }
```

```
    else
    {
        cout << "Ditemukan " << jumlahHasil << " data dengan rating " <<
                targetRating << ":" << endl;
        for (int i = 0; i < jumlahHasil; i++)
        {
            int indexAsli = hasilIndex[i];
            tampilDataSingkat(riwayat[indexAsli], indexAsli + 1);
        }
    }

    delete[] hasilIndex;
}
```

4. Hasil Output

```
==============================
   SELAMAT DATANG DI SLEEPY
==============================
1. Register (Daftar Akun)
2. Login
3. Keluar Program
Pilih: 1

--- REGISTER ---
Masukkan Username Baru: pares
Masukkan Password Baru: 045
Registrasi Berhasil! Silahkan Login.
```

Gambar 4.1 Output Register

```
--- LOGIN (Percobaan 1/3) ---
Username: pares
Password: 045
Login Berhasil! Halo, pares

=== APLIKASI SLEEP TRACKER (User: pares) ===
1. Tambah Catatan Tidur (Create)
2. Lihat Riwayat Tidur (Read - Tabel)
3. Ubah Catatan (Update)
4. Hapus Catatan (Delete)
5. Logout
Pilih:
```

Gambar 4.2 Output Login & Menu Utama

```
=== APLIKASI SLEEP TRACKER (User: pares) ===
1. Tambah Catatan Tidur (Create)
2. Lihat Riwayat Tidur (Read - Tabel)
3. Ubah Catatan (Update)
4. Hapus Catatan (Delete)
5. Logout
Pilih: 1

--- Tambah Data ---
Jam Tidur (HH:MM): 22:00
Jam Bangun (HH:MM): 07:00
Rating Kualitas (1-5): 5
Gangguan: tidka ada
Total Durasi (Jam): 9
Status Tidur (isi bebas): bagus
Data berhasil disimpan!
```

Gambar 4.3 Output Create

```
=== APLIKASI SLEEP TRACKER (User: pares) ===
1. Tambah Catatan Tidur (Create)
2. Lihat Riwayat Tidur (Read - Tabel)
3. Ubah Catatan (Update)
4. Hapus Catatan (Delete)
5. Logout
Pilih: 2

==========================================================================================
NO   WAKTU            RATING   DURASI(JAM)    STATUS      GANGGUAN
------------------------------------------------------------------------------------------
1    22:00-07:00      5        9.0            bagus       tidka ada
==========================================================================================
```

Gambar 4.4 Output Read

```
=== APLIKASI SLEEP TRACKER (User: pares) ===
1. Tambah Catatan Tidur (Create)
2. Lihat Riwayat Tidur (Read - Tabel)
3. Ubah Catatan (Update)
4. Hapus Catatan (Delete)
5. Logout
Pilih: 3

--- Ubah Data ---

==========================================================================================
NO   WAKTU            RATING   DURASI(JAM)    STATUS      GANGGUAN
------------------------------------------------------------------------------------------
1    22:00-07:00      5        9.0            bagus       tidka ada
==========================================================================================
Masukkan Nomor Data yang ingin diubah: 1
Masukkan data pengganti:
Jam Tidur (HH:MM): 23:00
Jam Bangun (HH:MM): 09:00
Rating Kualitas (1-5): 4
Gangguan: tidak ada
Total Durasi (Jam): 10
Status Tidur (isi bebas): bagus
Data Berhasil Diperbarui!
```

Gambar 4.5 Output Menu Update

```
=== APLIKASI SLEEP TRACKER (User: pares) ===
1. Tambah Catatan Tidur (Create)
2. Lihat Riwayat Tidur (Read - Tabel)
3. Ubah Catatan (Update)
4. Hapus Catatan (Delete)
5. Logout
Pilih: 4
Masukkan Nomor Data yang ingin dihapus: 1
Data Berhasil Dihapus!
```

Gambar 4.6 Output Menu Delete

```
Pilih: 5
Data berhasil diurutkan berdasarkan gangguan (A-Z).

==========================================================================================
NO   WAKTU            RATING   DURASI(JAM)    STATUS      GANGGUAN
------------------------------------------------------------------------------------------
1    19:00-07:00      5        9.0            Bugar       ada
2    07:00-19:00      5        9.0            Bugar       tidak ada
==========================================================================================
```

Gambar 4.7 Output Sorting Ascending Huruf

```
Pilih: 6
Data berhasil diurutkan berdasarkan durasi (besar-ke-kecil).

==========================================================================================
NO   WAKTU            RATING   DURASI(JAM)    STATUS      GANGGUAN
------------------------------------------------------------------------------------------
1    19:00-07:00      5        9.0            Bugar       ada
2    07:00-19:00      5        9.0            Bugar       tidak ada
==========================================================================================
```

Gambar 4.8 Output Sorting Descending Angka

```
Pilih: 7
Data berhasil diurutkan berdasarkan jam tidur (awal-ke-akhir).

==========================================================================================
NO   WAKTU            RATING   DURASI(JAM)    STATUS      GANGGUAN
------------------------------------------------------------------------------------------
1    07:00-19:00      5        9.0            Bugar       tidak ada
2    19:00-07:00      5        9.0            Bugar       ada
==========================================================================================
```

Gambar 4.9 Output Sorting Bebas

```
--- MENU SEARCHING ---
1. Cari Angka (Rating) [Sequential Search]
2. Cari Kata (Gangguan) [Binary Search]
3. Kembali ke Menu Utama
Pilih: 1
Masukkan rating yang dicari (1-5): 4
Ditemukan 1 data dengan rating 4:
Data ke-2 | Waktu: 20:00-08:00 | Rating: 4 | Durasi: 9.0 | Status: Bugar | Gangguan: tes2
```

Gambar 4.10 Output Searching Sequential

```
--- MENU SEARCHING ---
1. Cari Angka (Rating) [Sequential Search]
2. Cari Kata (Gangguan) [Binary Search]
3. Kembali ke Menu Utama
Pilih: 2
Masukkan gangguan yang dicari (harus sama persis): tes
Gangguan "tes" ditemukan pada 1 data:
Data ke-1 | Waktu: 19:00-08:00 | Rating: 5 | Durasi: 8.0 | Status: Bugar | Gangguan: tes
```

Gambar 4.10 Output Searching Binary

5. Langkah-langkah GIT

5.1 GIT Add

```
PS D:\0.PRAKTIKUM-APL'A2-2025> git add .
warning: in the working copy of 'post-test/post-test-apl-7/2509106045-ATHASYAHRI-SYAWAL-FAHREZY-PT-7.cpp', LF will be replaced by CRLF the next time Git touches it
```

Gambar 5.1 Git Add

Saya menjalankan git init, muncul seperti itu karena folder sudah pernah dijadikan repository Git sebelumnya, jadi Git hanya menginisialisasi ulang repo lama, bukan membuat yang baru.

5.2 GIT Commit

```
PS D:\0.PRAKTIKUM-APL'A2-2025> git commit -m "belum laporan"
[main ca5573e] belum laporan
 2 files changed, 804 insertions(+)
 create mode 100644 post-test/post-test-apl-7/2509106045-ATHASYAHRI-SYAWAL-FAHREZY-PT-7.cpp
 create mode 100644 post-test/post-test-apl-7/2509106045-ATHASYAHRI-SYAWAL-FAHREZY-PT-7.exe
```

Gambar 5.2 Git Commit

Git commit adalah perintah untuk menyimpan perubahan yang sudah masuk staging area ke dalam riwayat repository.

5.3 GIT Push

```
PS D:\0.PRAKTIKUM-APL'A2-2025> git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 497.33 KiB | 7.54 MiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/asyaress/praktikum-apl.git
   6254a32..ca5573e  main -> main
```

Gambar 5.3 Git Push

Saya menjalankan git push -u origin main, hasilnya adalah mengirimkan (upload) commit yang ada di branch main lokal ke GitHub (remote origin).